

1. Minimizing Delay

(a)

Construction: $D[i] = D$ for all i (equal deadlines).

In any ordering σ , the completion time of job $\sigma(k)$ is $C_{\sigma(k)} = \sum_{j=1}^k A[\sigma(j)]$.

$$\text{delay}(\sigma(k)) = C_{\sigma(k)} - D[\sigma(k)] = \sum_{j=1}^k A[\sigma(j)] - D$$

Since $C_{\sigma(1)} \leq C_{\sigma(2)} \leq \dots \leq C_{\sigma(n)}$, the maximum is always at $k = n$:

$$\max_k \text{delay}(\sigma(k)) = \sum_{i=1}^n A[i] - D$$

This value is independent of σ , so all $n!$ permutations are optimal. $\boxed{n!}$

(b)

Construction: $A[i] = 1$, $D[i] = i$ for all i .

Greedy (sort by D): $\sigma = \text{id}$. Completion $C_k = k$, delay $= k - k = 0 \forall k$.

$$\max_k \text{delay} = 0$$

Uniqueness: Any optimal σ must have $\max \text{delay} \leq 0$, i.e., $C_{\sigma(k)} - D[\sigma(k)] \leq 0 \forall k$. Since $C_{\sigma(k)} = k$:

$$k - \sigma(k) \leq 0 \implies \sigma(k) \geq k \quad \forall k$$

$$\sum_{k=1}^n \sigma(k) \geq \sum_{k=1}^n k = \frac{n(n+1)}{2}; \text{ but } \sigma \text{ is a permutation: } \sum_{k=1}^n \sigma(k) = \frac{n(n+1)}{2} \implies \sigma(k) = k \quad \forall k \implies \boxed{1}$$

(c)

Counterexample: $n = 2$, $A = [1, 10]$, $D = [100, 2]$.

i	1	2
$A[i]$	1	10
$D[i]$	100	2

Sort by duration: order 1, 2. Completions: 1, 11. Delays: $1 - 100 = -99$, $11 - 2 = 9$. Max = 9.

Sort by deadline (OPT): order 2, 1. Completions: 10, 11. Delays: $10 - 2 = 8$, $11 - 100 = -89$.

Max = 8. $\boxed{\text{Sort by duration} = 9 > 8 = \text{OPT}}$

2. Minimizing Wait Time

Let $L[i] = Q[i] - R[i]$ (duration of request i). In ordering σ , the actual start time is

$$S[\sigma(k)] = \sum_{j=1}^{k-1} L[\sigma(j)]$$

and the wait time is $S[\sigma(k)] - R[\sigma(k)]$. We minimize $\max_k \{S[\sigma(k)] - R[\sigma(k)]\}$.

Algorithm: Sort by $Q[i]$ (requested end time) in nondecreasing order.

```

begin
  MINWAIT( $R[1..n]$ ,  $Q[1..n]$ )
  Sort intervals so that  $Q[1] \leq Q[2] \leq \dots \leq Q[n]$ 
   $t \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
     $S[i] \leftarrow t$ 
     $t \leftarrow t + (Q[i] - R[i])$ 
  return  $S[1..n]$ 

```

Proof (exchange argument): Let σ be any ordering with adjacent positions $k, k+1$ occupied by jobs a, b with $Q[a] > Q[b]$. Let $P = \sum_{j=1}^{k-1} L[\sigma(j)]$ (cumulative duration before position k).

Before swap (order a, b):

$$\begin{aligned}
 W_a &= P - R[a] \\
 W_b &= P + L[a] - R[b]
 \end{aligned}$$

After swap (order b, a):

$$\begin{aligned}
 W'_b &= P - R[b] \\
 W'_a &= P + L[b] - R[a]
 \end{aligned}$$

Since $L[a] = Q[a] - R[a] \geq 0$: $W'_b = P - R[b] \leq P + L[a] - R[b] = W_b$.

Since $Q[b] < Q[a]$, i.e. $L[b] + R[b] < L[a] + R[a]$: $W'_a = P + L[b] - R[a] < P + L[a] - R[b] = W_b$.

$\therefore \max(W'_b, W'_a) \leq W_b \leq \max(W_a, W_b)$.

Positions outside $k, k+1$ are unaffected since $L[a] + L[b]$ is invariant under the swap. Each swap removes an inversion without increasing the max wait. After finitely many swaps we reach the sorted-by- Q ordering, whose max wait $\leq$ that of any other ordering. $\square$

Runtime: Sort $O(n \log n)$ + schedule $O(n) = \boxed{O(n \log n)}$

3. Elementary Graph Problems

(a)

G is a tree $\iff G$ is connected and $m = n - 1$.

```

begin
  ISTREE( $G = (V, E)$ )
  if  $m \neq n - 1$  then
    return false
  Run BFS from vertex 1; let  $r$  = number of visited vertices
  return  $r = n$ 

```

Correctness: If $m \neq n - 1$, G is not a tree. If $m = n - 1$ and G is connected (BFS visits all n vertices), then G is a tree. If BFS visits $< n$ vertices, G is disconnected, so not a tree.

Runtime: $O(n + m)$

(b)

Run DFS on G . Every non-tree edge (u, v) (u descendant, v ancestor) is a back edge forming a fundamental cycle of length $\text{depth}[u] - \text{depth}[v] + 1$.

```

begin
  HASEVENCYCLE( $G = (V, E)$ )
  Run DFS on  $G$ ; record  $\text{depth}[v]$ ,  $\text{parent}[v]$ ; collect back edges  $B$ 
  // Phase 1: check for even fundamental cycle
  for each  $(u, v) \in B$  do
    if  $\text{depth}[u] - \text{depth}[v] + 1$  is even then
      return true
  // Phase 2: check for doubly-covered tree edge
   $\text{diff}[1..n] \leftarrow 0$ 
  for each  $(u, v) \in B$  do
     $\text{diff}[u] \leftarrow \text{diff}[u] + 1$ 
     $\text{diff}[v] \leftarrow \text{diff}[v] - 1$ 
  for each vertex  $v$  in post-order do
     $\text{sub}[v] \leftarrow \text{diff}[v] + \sum_{c \in \text{children}(v)} \text{sub}[c]$ 
    if  $\text{sub}[v] \geq 2$  then
      return true
  return false

```

Correctness:

Phase 1: An even-length fundamental cycle is itself an even cycle.

Phase 2: $\text{sub}[v]$ counts the number of back edges whose fundamental cycle includes the tree edge $(\text{parent}[v], v)$. If $\text{sub}[v] \geq 2$, two odd fundamental cycles share this tree edge. Their shared edges

form a contiguous tree path, so the symmetric difference is a single cycle of length

$$|C_1| + |C_2| - 2k \quad (\text{odd} + \text{odd} - \text{even} = \text{even})$$

Converse: If neither condition holds, every tree edge is covered by ≤ 1 back edge, so no two fundamental cycles share an edge. Every cycle in G is a symmetric difference of fundamental cycles; with no shared edges this is a disjoint union of odd cycles, so no even cycle exists. $\square$

Runtime: DFS $O(n + m)$ + Phase 1 $O(m)$ + Phase 2 $O(n + m)$ = $\boxed{O(n + m)}$

(c)

```

begin
  COUNTTRIANGLES( $G = (V, E)$ )
  Build adjacency matrix  $M[1..n][1..n]$  and adjacency lists
  count  $\leftarrow 0$ 
  for  $u \leftarrow 1$  to  $n$  do
    for each  $v \in \text{adj}[u]$  with  $v > u$  do
      for each  $w \in \text{adj}[u]$  with  $w > v$  do
        if  $M[v][w] = 1$  then
          count  $\leftarrow$  count + 1
  return count

```

Correctness: Each triangle $\{u, v, w\}$ with $u < v < w$ is counted exactly once, at vertex u .

Runtime:

Per vertex u : $O(\deg(u)^2)$ pair checks, each $O(1)$ via matrix

$$\text{Total: } \sum_u \deg(u)^2 \leq \max(\deg) \cdot \sum_u \deg(u) \leq (n-1) \cdot 2m \implies \boxed{O(nm)}$$

4. Classifying Roundworms

Model the data as a graph $G = (V, E)$ with $V = \{1, \dots, n\}$. Each edge (i, j) carries a label $\ell_{ij} \in \{S, D\}$ (same or different subspecies). The data is consistent with two subspecies $\iff \exists$ a 2-coloring $c : V \rightarrow \{A, B\}$ with $\ell_{ij} = S \implies c(i) = c(j)$ and $\ell_{ij} = D \implies c(i) \neq c(j)$.

```

begin
  CLASSIFYROUNDWORMS( $n$ , edges with labels)
  Build adjacency list with labels
  color[1.. $n$ ]  $\leftarrow$  nil
  for  $s \leftarrow 1$  to  $n$  do
    if color[ $s$ ] = nil then
      color[ $s$ ]  $\leftarrow$   $A$ ; enqueue  $s$ 
      while queue  $\neq \emptyset$  do
         $u \leftarrow$  dequeue
        for each  $(u, w, \ell)$  in adjacency list do
          expected  $\leftarrow$  if  $\ell = S$  then color[ $u$ ]
          else opposite of color[ $u$ ]
          if color[ $w$ ] = nil then
            color[ $w$ ]  $\leftarrow$  expected; enqueue  $w$ 
          else if color[ $w$ ]  $\neq$  expected then
            return inconsistent
  return consistent

```

Correctness: Within each connected component, BFS forces a unique coloring from the source. If no conflict arises, the constructed coloring satisfies all constraints $\implies$ consistent. If a conflict occurs at edge (u, w) , two paths from the source to w impose contradictory colors, so $\forall$ 2-colorings at least one constraint is violated $\implies$ inconsistent. $\square$

Runtime: BFS visits each vertex once and each edge once. $\boxed{O(n + m)}$